When using foreach loops, making changes to elements requires a reference:

```
$arr = [1, 2, 3];
foreach ($arr as $ele) {
    $ele++;
}
print_r($arr);

foreach ($arr as &$ele) {
    $ele++;
}
print_r($arr);

Output:
Array
(
    [0] => 1
    [1] => 2
    [2] => 3
)
Array
(
    [0] => 2
    [1] => 3
    [2] => 4
)
```

In the first loop – `foreach ($arr as $ele)` – changing the iterator `$ele` had no effect on the original array. This is because `$ele` is treated as a new value if it is written to (PHP's copy-on-write behavior; see Chapter 3, "Functions," for a detailed explanation). If we want to make changes to the original elements, we have to add a reference operator `&` to the iterator variable when it is declared in the `foreach` expression `foreach ($arr as &$ele)`. With the second loop, the values in the original array are changed.